

Rationale Generation for Performance-Improving Code Changes

Luca Francesco Macera

June 13, 2026

Contents

1	Introduction	2
2	Model Selection	2
3	Pipeline Definition	2
4	Results	4
5	Conclusion	5

1 Introduction

Software performance is a critical dimension of code quality, yet understanding *why* a particular change makes code faster is rarely straightforward. A developer may recognise that a pull request improves throughput, but articulating the underlying mechanism requires careful analysis and domain knowledge.

In this report we will show how we can instruct a large language model (LLM) to accurately generate a detailed code performance and quality analysis given a set of code changes on a codebase. We ground the study in the **SWE-Perf** dataset [1], a curated benchmark of 140 real-world performance-improving commits drawn from nine popular Python libraries, including `pydata/xarray`, `scikit-learn`, and `sympy`. Each instance pairs a Git diff with repeated timing measurements, providing an objective signal of improvement.

Our approach consists of two main components. First, we select and configure a language model capable of reasoning over code, commit history, and benchmark data. Second, we define a structured analysis pipeline that guides the model through a sequence of information-gathering and reasoning steps (fetching commit metadata, running static pattern analysis, retrieving GitHub context, and performing semantic analysis) before producing the final rationale in a fixed output format.

2 Model Selection

The LLM we've chosen to use is Anthropic's **Claude Sonnet v4.6**. Compared to other alternatives (like OpenAI GPT 5.4, or Google Flash 3.1), its large context window (1 million tokens) helps in gathering lots of information without the need to compact the conversation or cutting out important details which, in the context of this report, are essential to fully capture the reasons and the consequences of the changes implemented on the codebase. Furthermore, like all other Anthropic products, it supports an easy way of customizing the behaviour of the model using simple markdown files like the global `CLAUDE.md` file. We can easily tell the agent how it should respond to the user and what it should execute in order to answer with the correct data without relying on the "black-box" logic of the base LLM. We can also add custom "skills" which enables the model to do specific operations without explicit user instruction on each run. These two things are possible with other LLMs but are harder to implement and require the user to copy and paste all the custom behaviours and configurations every time on a new conversation. This is not the case with Claude as all the custom markdown files are automatically picked up by the agent on start up and can be used by the model right away as if they are part of the original agent's capabilities.

3 Pipeline Definition

The rationale generation pipeline is defined entirely in the project's `CLAUDE.md` file, which the agent loads automatically at startup. The pipeline accepts two types of input:

- **Dataset reference:** an index or instance ID from the SWE-Perf CSV. All five steps are executed.

- **GitHub commit reference:** a repository name and commit hash (or URL). Step 1 is skipped and the pipeline begins directly at step 2, using the commit as the sole entry point.

No step may be silently skipped: if a step fails, the agent is required to document the failure and continue with the remaining steps.

1. **Load dataset fields** (*dataset input only*). The agent loads the target instance from the dataset CSV and extracts all available fields: repository, patch, test patch, problem statements, efficiency tests, timing data, and commit hashes. The `duration_changes` field (a JSON array of repeated base/head timing measurements) is parsed and per-run speedups are computed for every test case.
2. **Fetch commit information.** Using the GitHub API, the agent retrieves the commit associated with the change and checks whether it is a merge commit (i.e., has multiple parents). If it is, all individual commits bundled in the merge are retrieved and analysed separately, since merge messages are typically uninformative and the relevant optimisation may be buried in one of many commits.
3. **Analyse code patterns.** The patch is passed to a static pattern analyser (a custom Python script the agent is instructed to invoke via the shell) which attempts to classify the change into one of several known patterns: caching/memoisation, library function substitution, data structure change, early termination, redundancy removal, or algorithm change. This output is treated as a starting point only (the agent is explicitly instructed not to rely on it as the final classification).
4. **Fetch GitHub context.** The agent retrieves the full source file(s) modified by the patch, as well as any related issues or pull request discussions referenced in the commit message. This provides surrounding code context that is not present in the diff alone and helps establish the motivation behind the change and may reveal impacts on unchanged parts of the codebase.
5. **Semantic analysis.** With all data gathered, the agent reasons explicitly about what computation existed before and after the change, what work is eliminated or made more efficient, and whether the performance improvement was the primary intent or an incidental side effect. The change is classified into one of four categories: *targeted optimization*, *side effect of bug fix*, *ambiguous*, or *not a performance change*.

The output is written in a fixed, section-by-section format that covers the problem being solved, the classification, the rationale for the specific optimisation chosen, any behavioural side effects, a statistical summary of the benchmark data, and a code quality assessment. All performance figures reported in the rationale must be traceable to a concrete source (typically the `duration_changes` field) and the rationale must never reference the dataset as a whole (e.g., relative rankings across instances).

The user can always add their own requests via the initial prompt, for example changing the structure of the output or how the classification is done, enabling the user to always have control over what the LLM outputs. What the user cannot do is change the core logic of the pipeline, like skipping steps or ignoring the data gathered by a specific step. These are the pillars needed to generate the final rationale and are designed specifically to create the context needed to fully capture the consequences of the changes and how they affect the performance and quality of the application. Without any one of them, the agent may miss important details or insights, inevitably degrading the final reasoning.

4 Results

The pipeline was applied to all 140 instances of the SWE-Perf dataset. Table 1 summarises the classification breakdown produced by the agent.

Classification	Count	Percentage
Side effect of bug fix	80	57.1%
Not a performance change	43	30.7%
Targeted optimization	16	11.4%
Ambiguous	1	0.7%
Total	140	100%

Table 1: Classification breakdown across the 140 SWE-Perf instances.

The dominant category is *side effect of bug fix* (57.1%), which aligns with the generally low performance metrics observed across the dataset: the majority of changes were made to correct incorrect behaviour, with any timing improvement being incidental rather than intentional. The second largest category, *not a performance change* (30.7%), covers instances such as documentation updates, API additions, and aesthetic fixes. The agent correctly identifies these by combining commit message analysis with the benchmark data (the median `human_performance` value for this group is 0.0057, consistent with pure measurement noise rather than a genuine speedup). 16 instances (11.4%) were classified as *targeted optimizations*, i.e. changes made with the explicit intent of improving performance. These show a higher mean improvement (0.23) compared to side-effect cases (0.13), consistent with the expectation that deliberate optimisations tend to yield larger gains. A single instance was classified as *ambiguous*, discussed in detail in the next section.

The Ambiguous Case: `pydata__xarray-9881`

The sole ambiguous instance involves a rewrite of the interpolation logic in the `xarray` library. The change was motivated by two distinct problems reported by users: a correctness issue where certain interpolation calls failed entirely, and a performance issue where the same operation became prohibitively slow on large datasets. Because both problems originated from the same design flaw in the original implementation, a single architectural rewrite resolved both at once.

This is precisely what makes classification difficult. The change is not a pure bug fix — the author explicitly referenced the performance problem and included visual evidence of the improvement in the pull request. But it is not a pure targeted optimization either, since the correctness fix was equally necessary and the PR would have been merged on those grounds alone.

The benchmark results reinforce the ambiguity: the performance improvement is large (up to 90% on the affected use cases) but completely absent on simpler, unaffected ones — a pattern consistent with fixing a specific broken path rather than broadly optimizing the codebase. The agent correctly identified this case as ambiguous on two independent runs, concluding that neither standard label fully captures the intent of the change.

Consistency across generations

A key concern with LLM-based pipelines is output stability: the same input may produce substantially different outputs across runs. In practice, the agent exhibits strong consistency when prompted on the same instance multiple times. The core content of the rationale (the classification, the identified optimisation pattern, and the performance analysis) remains stable across generations. Variation is limited to surface-level differences in phrasing and layout, which do not affect the analytical conclusions. This stability is a direct consequence of the structured pipeline: by anchoring every reasoning step to concrete, deterministic data sources (commit metadata, static analysis output, benchmark figures), the agent has little room to drift between runs.

5 Conclusion

This report presented a structured approach to automated rationale generation for performance-improving code changes, applied to the SWE-Perf benchmark dataset. By combining a carefully configured large language model with a mandatory, multi-step analysis pipeline, we were able to produce consistent, evidence-based explanations for all 140 instances in the dataset.

The results reveal an important characteristic of the dataset itself: the majority of changes (57.1%) are bug fixes that improve performance incidentally, and a further 30.7% are not performance changes at all. Only 11.4% of instances represent deliberate, targeted optimisations, and a single instance resisted classification entirely, as discussed in Section 4. This distribution highlights a broader challenge in performance engineering: performance improvements rarely arrive in isolation and are often entangled with correctness fixes and architectural refactors, making classification non-trivial even for human reviewers.

The pipeline proved effective at navigating this complexity. By grounding each reasoning step in deterministic data sources (commit metadata, static analysis, benchmark figures, and GitHub context), the agent consistently produces accurate and stable classifications across multiple generations. The structured format also ensures that every performance claim is traceable and that no step can be silently skipped, which are the two properties most critical for producing trustworthy rationales at scale.

A key lesson from this work is that prompting an LLM via a well-designed system file (rather than fine-tuning or retrieval augmentation) is sufficient to produce reliable, structured analytical output, provided the pipeline anchors the model’s reasoning to external, verifiable data at every step.

References

- [1] Xinyi He, Qian Liu, Mingzhe Du, Lin Yan, Zhijie Fan, Yiming Huang, Zejian Yuan, and Zejun Ma. SWE-Perf: Can language models optimize code performance on real-world repositories? *arXiv preprint arXiv:2507.12415*, 2025.